

DATA ANALYTICS & BUSINESS INTELLIGENCE PROJECT

Product Performance Analytics on Marketplace Data

การวิเคราะห์ประสิทธิภาพสินค้าจากข้อมูลรีวิว Amazon หมวด Beauty & Personal Care — ตั้งแต่ ELT Pipeline, Star Schema, การวิเคราะห์และโมเดล Machine Learning ไปจนถึง Interactive Dashboard

สมาชิกในทีม

jjentaa · hoggyna · Krittiphong Praichuen

(ชื่อ/บัญชีอ้างอิงจาก git commit history — โปรดยืนยันหรือแก้ไขเป็นชื่อ-นามสกุลเต็มก่อนส่งจริง)

Repository: github.com/jjentaa/Product-Performance-Analytics-on-Marketplace-Data

Dataset ต้นทาง: McAuley-Lab/Amazon-Reviews-2023 (Hugging Face)

จัดทำ: สิงหาคม 2026

1 โจทย์ธุรกิจ (Business Problem) และ Stakeholders

บริบทของโปรเจกต์

โปรเจกต์นี้เป็นงานวิเคราะห์ข้อมูลเชิงวิชาการ (ไม่ได้ผูกกับองค์กรหรือลูกค้าจริง) ที่จำลอง สถานการณ์การทำงานของทีมวิเคราะห์ข้อมูล (Data/Analytics team) ในแพลตฟอร์มมาร์เก็ตเพลส โดยใช้ข้อมูลรีวิวสาธารณะของ Amazon (Amazon Reviews 2023) เป็นกรณีศึกษา เพื่อสาธิต กระบวนการทำงานแบบครบวงจร ตั้งแต่การดึงและแปลงข้อมูล (ELT) การออกแบบคลังข้อมูล (Star Schema) การวิเคราะห์เชิงสถิติและการสร้างโมเดล Machine Learning ไปจนถึงการนำเสนอผ่าน Interactive Dashboard

Business Problem

สินค้าในหมวด **Beauty & Personal Care** บนมาร์เก็ตเพลสมีจำนวนมากและมีรีวิวสะสม หลักล้านรายการ แต่ข้อมูลดิบอยู่ในรูปแบบที่ไม่พร้อมใช้งาน (JSONL กระจายเป็นไฟล์รายหมวด ข้อความมี HTML ปนเปื้อน ราคาสินค้าขาดหายไปส่วนหนึ่ง) ทำให้ทีมธุรกิจไม่มีเครื่องมือสรุป ภาพรวมคะแนนสินค้า ตรวจสอบสินค้าที่มีปัญหา หรือประเมินความน่าเชื่อถือของรีวิวได้อย่าง รวดเร็วและน่าเชื่อถือ ส่งผลให้การตัดสินใจด้านสินค้า (จัดซื้อ คุณภาพ การตั้งราคา) ล่าช้าและเสี่ยงต่อการเชื่อถือรีวิวที่ถูกต้อง

เป้าหมายของโปรเจกต์คือสร้าง **pipeline ข้อมูลที่เชื่อถือได้** (ตรวจสอบคุณภาพข้อมูล อัตโนมัติทุกขั้นตอน) และ **dashboard เชิงโต้ตอบ** ที่ตอบคำถามธุรกิจสำคัญได้ครบถ้วน ทั้งจากการวิเคราะห์เชิงสถิติล้วน ๆ และจากโมเดล Machine Learning

Stakeholders (สมมติเป็นบทบาทประกอบ เพื่อกำหนดขอบเขตคำถามธุรกิจ)

บทบาท	ความต้องการ	คำถามที่เกี่ยวข้อง
Category / Product Manager	ต้องการรู้ว่าแบรนด์ไหนแข็งแกร่ง สินค้าไหนมีปัญหาต้องแก้ก่อน และควรตั้งราคาในช่วงใด	A1, A2, A3, B1, B3
Trust & Safety / Customer Experience	ต้องการติดตามความพึงพอใจลูกค้า ตรวจสอบรีวิวปลอม/รีวิวจ้าง และรู้ว่าลูกค้าบ่นเรื่องอะไร	A5, B2, B4, B5
Marketing / Content Team	ต้องการรู้ว่ารีวิวแบบไหนมีประโยชน์ต่อผู้ซื้อ เพื่อออกแบบกลไกกระตุ้นการเขียนรีวิว	A4
Data / Analytics Team	ต้องการ pipeline ที่ตรวจสอบคุณภาพได้เอง และ dashboard ที่ดูและรักษาต่อได้	ทุกข้อ (โครงสร้างระบบ)

ขอบเขตข้อมูล — ไฟล์ 3 หมวดย่อยที่รวมเป็นหมวดใหญ่เดียวกันได้อย่างมีความหมาย คือ **All_Beauty, Amazon_Fashion** และ **Health_and_Personal_Care** (สินค้าที่ผู้บริโภคใช้กับร่างกาย/รูปลักษณ์ของตนเอง) ทำให้เปรียบเทียบข้ามหมวดได้อย่างมีประสิทธิภาพ

2 Business Questions

แบ่งเป็น 2 กลุ่มตามวิธีตอบ — **กลุ่ม A** ตอบได้ด้วยการนับ/ค่าเฉลี่ยบน star schema โดยตรง ไม่ต้องสร้างโมเดล เพราะเป็นคำถาม ที่ "บรรยายสิ่งที่เกิดขึ้นไปแล้ว" และ **กลุ่ม B** ต้องสร้างโมเดล เพราะต้องอ่านข้อความอิสระ ทำนายอนาคต หรือหาโครงสร้างที่ซ่อนอยู่ ซึ่ง ไม่มีคอลัมน์ใดตอบตรง ๆ ได้ (รายละเอียด SQL และวิธีตั้งโจทย์โมเดลฉบับเต็มอยู่ที่ [docs/business_questions.md](#))

กลุ่ม A — ตอบได้จากการ Analyze (ไม่ใช่โมเดล)

#	คำถาม	ใช้ตอบอะไร
A1	แบรนด์ไหนแข็งแกร่งที่สุดในแต่ละหมวดย่อย และควรใช้ใครเป็น benchmark?	เส้นฐานเปรียบเทียบผลงานแบรนด์
A2	ราคาแพงขึ้นแล้วลูกค้าพอใจขึ้นจริงไหม และมีช่วงราคาไหนที่ลูกค้าผิดหวังเป็นระบบ?	กลยุทธ์การตั้งราคา
A3	สินค้าตัวไหนกำลังขายลง — คะแนนช่วงหลังตกจากค่าเฉลี่ยตลอดชีพ?	สัญญาณเตือนล่วงหน้า (early warning)
A4	รีวิวแบบไหนที่คนซื้ออ่านแล้วบอกว่ามีประโยชน์?	ออกแบบการจัดลำดับรีวิว/กระตุ้นการเขียนรีวิว
A5	3 หมวดย่อยต่างกันอย่างไร และคะแนนของหมวดไหนน่าเชื่อถือน้อยที่สุด?	เงื่อนไขการเทียบข้อมูลข้ามหมวด

กลุ่ม B — ต้องสร้างโมเดล

#	คำถาม	เทคนิค
B1	สินค้าใหม่ที่จะรับเข้ามาขาย มีแนวโน้มจะได้คะแนนเท่าไร?	Regression
B2	ข้อความในรีวิวตรงกับจำนวนดาวที่ให้ไหม?	Classification (NLP)
B3	สินค้าในพอร์ตแบ่งได้เป็นกี่กลุ่มตามลักษณะ performance?	Clustering
B4	ลูกค้าบ่นเรื่องอะไรบ้างในรีวิวเชิงลบ และแต่ละหมวดบ่นต่างกันไหม?	Topic Modeling
B5	รีวิวไหนน่าสงสัยว่าเป็นรีวิวปลอมหรือรีวิวจ้าง?	Anomaly Detection

หลักการแบ่งกลุ่ม — ถ้าคำถามตอบได้ด้วย **GROUP BY** บนคอลัมน์ที่มีอยู่แล้ว จัดเป็นกลุ่ม A แต่ถ้าต้องเรียนรู้รูปแบบจากข้อมูล (เช่น ความหมายของข้อความ, การจัดกลุ่มที่ไม่มีป้ายกำกับ, การทำนายสิ่งที่ยังไม่เกิดขึ้น) จัดเป็นกลุ่ม B ทีมงานตอบคำถามชุดนี้ผ่านสอง เส้นทางคู่ขนาน: [analytics/analytics.ipynb](#) และ [analytics/Answer_Analytic.ipynb](#) (วิเคราะห์ล้วน, ไม่ใช่โมเดล) และ [model/model.ipynb](#) (B1–B5 ครบ) ผลลัพธ์ทั้งหมดถูกรวบรวมมาแสดงใน Dashboard เดียว

3 Measures สำหรับวัด Business Performance

Dashboard แสดง Measures หลัก 4 ตัวที่หัวหน้าหน้าเสมอ (อัปเดตตาม filter ที่เลือก) และ Measures รองอีก 2 ตัวที่ใช้ประกอบการวิเคราะห์เชิงลึกในแต่ละคำถาม

Measures หลัก (แสดงเป็น KPI card uu Dashboard)

จำนวนรีวิว (Total Reviews)	คะแนนเฉลี่ย (Avg Rating)	สัดส่วนรีวิวเชิงลบ (Negative Share)	สัดส่วนซื้อจริง (Verified Share)
3,658,936	3.97 / 5	19.5%	92.5%

Measure	นิยาม	ใช้ตอบคำถามไหน
Total Reviews	จำนวนรีวิวทั้งหมดหลังผ่านการทำความสะอาด (dedupe + กรองแถวเสีย) — ตัวแทนคร่าว ๆ ของปริมาณติมาณต์ (ไม่ใช่ยอดขาย เพราะชุดข้อมูลไม่มีข้อมูลยอดขาย)	A3, A5, แนวโน้มตามเวลา
Average Rating	คะแนนเฉลี่ย 1–5 ดาวของรีวิวทั้งหมดในขอบเขตที่เลือก — ตัวชี้วัดความพึงพอใจหลัก	A1, A2, A3, A5
Negative Review Share	สัดส่วนรีวิวที่ให้คะแนน ≤ 2 ดาว — ตัวชี้วัดความไม่พอใจ วัตถุประสงค์ปัญหาสินค้ามากกว่า ค่าเฉลี่ยรวม	A1, A3, B5
Verified Purchase Share	สัดส่วนรีวิวที่มาจากการซื้อจริง (ผ่านระบบของแพลตฟอร์ม) — ตัวชี้วัดความน่าเชื่อถือ ของฐานรีวิว	A5, B5
Polarized Share (รอง)	สัดส่วนรีวิวที่ให้คะแนน 1 หรือ 5 ดาว (ความเห็นสุดขั้ว) — ใช้วัดว่าคะแนนรวมถูกครอบงำด้วยความเห็นแบบรัก-หรือ-เกลียดมากแค่ไหน	A5, Insight #1
Share with Helpful Vote (รอง)	สัดส่วนรีวิวที่ได้รับโหวต "มีประโยชน์" อย่างน้อย 1 เสียง — ใช้แทน average helpful_vote เพราะข้อมูลจริงเบ้ขวาทัก (ส่วนใหญ่ได้ 0 โหวต)	A4

ข้อจำกัดสำคัญ — ชุดข้อมูลนี้ **ไม่มีข้อมูลยอดขายหรือรายได้** Measures ทั้งหมด จึงสะท้อนเฉพาะ "คะแนนและพฤติกรรมการรีวิว" เท่านั้น ห้ามตีความจำนวนรีวิวเป็นยอดขายหรือ ส่วนแบ่งตลาดโดยตรง (ดูรายละเอียดเพิ่มเติมในหัวข้อ 5 และ Dashboard แท็บ "ข้อจำกัดของข้อมูล")

4 รายละเอียดและแหล่งที่มาของข้อมูล

แหล่งข้อมูลต้นทาง

ใช้ชุดข้อมูล **Amazon Reviews 2023** จัดทำโดย McAuley Lab, UC San Diego เผยแพร่แบบสาธารณะบน Hugging Face ([McAuley-Lab/Amazon-Reviews-2023](#)) ประกอบด้วยรีวิวสินค้าและ metadata สินค้า แยกเป็นไฟล์ JSONL รายหมวดสินค้า (33 หมวดใหญ่)

ขอบเขตที่เลือกใช้

หมวดย่อย	ขนาดไฟล์ดิบ	รีวิวหลัง ELT	สินค้า	เหตุผลที่เลือก
All_Beauty	~0.54 GB	694,252	112,565	เครื่องสำอาง/ผลิตภัณฑ์ความงาม ชื่อซ้ำบ่อย
Amazon_Fashion	~2.47 GB	2,475,694	825,869	ปัญหาไซส์/ความพอดี อัตรารีวิวสูงสุด
Health_and_Personal_Care	~0.35 GB	488,990	60,274	ของใช้ดูแลสุขภาพส่วนบุคคล
รวม	~3.36 GB	3,658,936	998,708	—

คอลัมน์สำคัญของข้อมูลดิบ

ไฟล์รีวิว	ไฟล์ metadata สินค้า
rating, title, text, images, asin, parent_asin, user_id, timestamp, helpful_vote, verified_purchase	title, main_category, average_rating, rating_number, price, store, categories, details, parent_asin

ช่วงเวลาและปริมาณข้อมูล

รีวิวครอบคลุมปี ค.ศ. 2000–2023 โดยปริมาณกระจุกตัวหนาแน่นในช่วงปี 2014–2021 (สูงสุดที่ปี 2019 ~512,000 รีวิว) และลดลงอย่างมากในปี 2022–2023 ซึ่งเป็นข้อจำกัดของ การเก็บข้อมูลต้นทาง ไม่ใช่ปริมาณที่ลดลงจริง (ดูหัวข้อ 5)

Dataset ที่จัดทำและเผยแพร่ต่อ (Hugging Face, public)

Dataset	เนื้อหา	ขนาด
Madnesss/amazon-beauty-star-schema	Star schema ที่ผ่าน ELT — <code>fact_review</code> , <code>dim_product</code> , <code>dim_user</code> , <code>dim_date</code> , <code>review_text</code>	~535 MB (Parquet)
Madnesss/amazon-beauty-analytics-ready	ข้อความล้างแล้ว + feature + label + แบ่ง train/val/test ตามเวลา	~656 MB (Parquet)

5 ปัญหาคุณภาพข้อมูลและวิธีแก้ไข

ปัญหา	ผลกระทบถ้าไม่แก้	วิธีแก้ไขที่ใช้จริง
HTML ปนในข้อความรีวิว (<code>
</code> , <code>&#34;</code> , <code>&amp;</code> ;)	Tokenizer บั๊ก tag เป็นคำ ทำให้ผลลัพธ์ NLP (B2, B4) เพี้ยน	เขียน SQL macro <code>clean_text()</code> ถอด HTML tag และ entity ทั้งหมด โดยถอด <code>&amp;</code> เป็นลำดับสุดท้าย เพื่อไม่ให้ entity ที่ escape ซ้อนสองชั้นแปลงผิด
รีวิวซ้ำ (สินค้าเดียวปรากฏในหลายไฟล์หมวด)	นับรีวิวซ้ำ ทำให้สถิติ (จำนวน/คะแนนเฉลี่ย) คลาดเคลื่อน	Dedupe ด้วยคีย์ (<code>user_id</code> , <code>asin</code> , <code>timestamp</code>) เก็บแถวที่ <code>helpful_vote</code> สูงสุด
Timestamp ผิดปกติ (นอกช่วงปี 1996–2024)	ปีข้อมูลเพี้ยน กระบกรภาพแนวโน้มตามเวลา และการแบ่ง train/test	กรองแถวที่ timestamp อยู่นอกช่วงที่สมเหตุสมผลออกในชั้น staging
ราคาสินค้าขาดหาย 79% แบบไม่สุ่ม (MNAR)	สินค้าที่รู้ราคามีคะแนนเฉลี่ยสูงกว่ากลุ่มไม่รู้ราคาอยู่ +0.22 ดาว — การวิเคราะห์ราคาโดยไม่ระวังจะได้ผลเอนไปทางบวก	คำนวณและแสดง missingness bias เป็น KPI ในตัว Dashboard (แท็บ Q3) พร้อมคำเตือนทุกครั้งที่มีการวิเคราะห์เรื่องราคา
สินค้าที่มีรีวิวแต่ไม่มี metadata	Fact table มีแถวกำพร้า (orphan) เพราะ join กับ <code>dim_product</code> ไม่ติด	สร้างแถว placeholder ใน <code>dim_product</code> พร้อมธง <code>has_metadata = false</code>
รีวิวสั้นเกินไป/ว่างเปล่าหลังล้าง (เช่น "ok", "👍")	เพิ่ม noise ให้โมเดล NLP โดยไม่ได้ให้ข้อมูลที่มีความหมาย	กรองรีวิวที่เหลือน้อยกว่า 3 คำออกจาก dataset #2 (preprocessing) เท่านั้น — dataset #1 ยังเก็บครบเพื่อความโปร่งใส
ปริมาณรีวิวปี 2022 เป็นต้นไปเก็บไม่ครบ (~468K ปี 2021 → ~58K ปี 2023)	เข้าใจผิดว่าตีमतกำลังหด และการแบ่ง train/val/test แบบเดิม (2022/2023) จะได้ test set แค่อะ ~1.7% ของข้อมูล	เลื่อนเส้นแบ่ง train/val/test เป็นปี 2021/2022 (ได้สัดส่วน 77/14/10) และติดป้ายเตือนถาวรในทุกที่ที่แสดงกราฟรายปี
คะแนนไม่สมดุล (4–5 ดาวครองส่วนใหญ่ของข้อมูล)	โมเดล Classification (B2) ทยอยคลาสเดียวตลอดก็ยิ่งดูมี accuracy สูง	ใช้ <code>class_weight='balanced'</code> ตอนเทรน และวัดผลด้วย macro-F1 แทน accuracy

ทุก stage ของ ELT pipeline มี **automated quality-check view** (10 การตรวจสอบ ต่อ stage เช่น referential integrity, คำซ้ำ, คำนอกช่วงที่เป็นไปได้) — ถ้าตรวจไม่ผ่าน ชั้น export จะหยุดทำงานทันที ไม่ปล่อยข้อมูลเสียออกไปสู่ dataset ปลายทาง

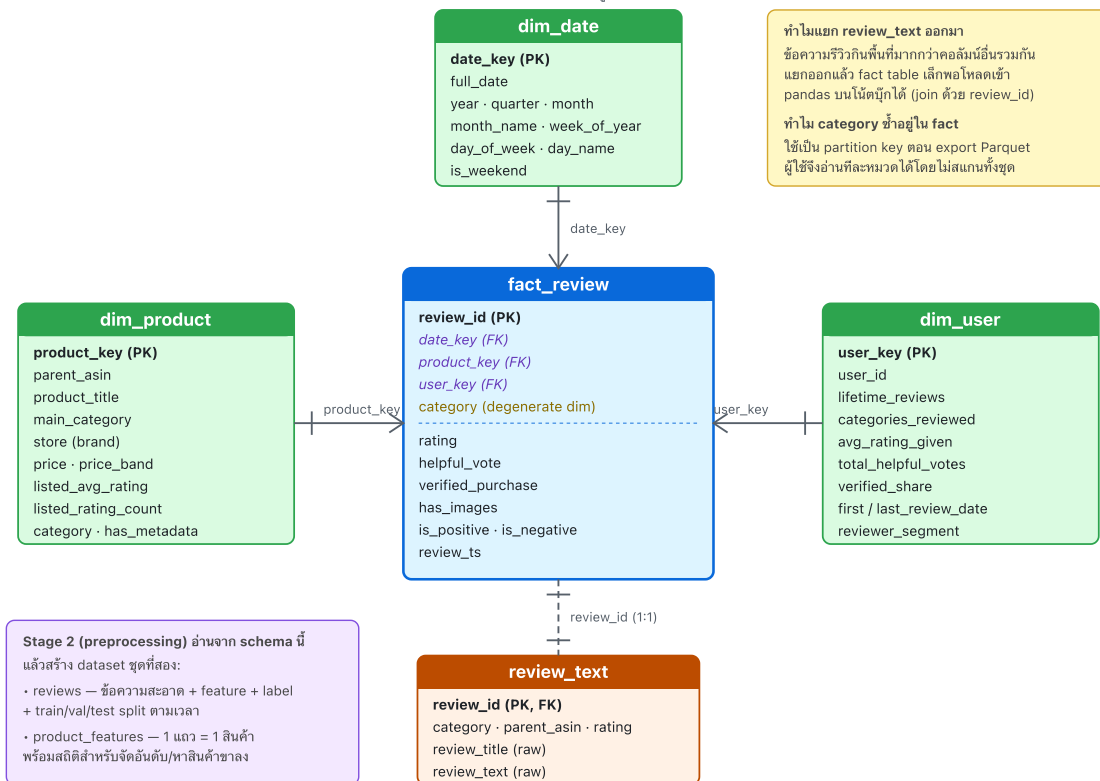
6 Star Schema

Grain ของ Fact Table: 1 แถว = 1 รีวิว — ออกแบบเป็น star schema แบบมาตรฐาน มี 1 fact table, 3 dimension tables และ 1 ตารางเสริมที่แยกข้อความออกมาต่างหาก

Amazon Reviews 2023 — Beauty & Personal Care

Star Schema — grain: 1 row = 1 review

3.66M รีวิว · 999K สินค้า · 2.92M ผู้รีวิว · ปี 2000–2023



สรุปตาราง

ตาราง	ชนิด	Grain / Primary Key	คอลัมน์สำคัญ
fact_review	Fact	1 แถว = 1 รีวิว review_id	date_key, product_key, user_key (FK), category (degenerate dim), rating, helpful_vote, verified_purchase, has_images, is_positive/is_negative, review_ts
dim_product	Dimension	1 แถว = 1 สินค้า product_key	parent_asin, product_title, main_category, store (แบรนด์), price, price_band, listed_avg_rating, has_metadata
dim_user	Dimension	1 แถว = 1 ผู้รีวิว user_key	lifetime_reviews, categories_reviewed, avg_rating_given, verified_share, reviewer_segment
dim_date	Dimension	1 แถว = 1 วัน date_key	full_date, year, quarter, month, week_of_year, day_name, is_weekend
review_text	ตารางเสริม	1 แถว = 1 รีวิว review_id (FK 1:1)	review_title, review_text (ข้อความดิบ) — แยกออกจาก fact เพื่อให้ fact table เล็กพอโหลดเข้า pandas ได้สบาย

เหตุผลออกแบบสั้น ๆ — แยก review_text ออกจาก fact เพราะข้อความกินพื้นที่มากกว่าคอลัมน์อื่นรวมกัน · category เข้าใน fact (degenerate dimension) เพื่อใช้เป็น partition key ตอน export Parquet · dim_product มีแถว placeholder (has_metadata=false) กันแถวถ้าปรา · dim_user คำนวณจากขอบเขต 3 หมวดนี้เท่านั้น ไม่ใช่ประวัติทั้งหมดของผู้ใช้บน Amazon

7 ขั้นตอนและเครื่องมือที่ใช้ทำ ETL

สถาปัตยกรรมโดยรวม (4 stage)

- 1 **ELT (`elt/`)** — Extract: ดาต้าโหลด JSONL จาก Hugging Face → Load: นำเข้า DuckDB schema `raw` ตามสภาพเดิม (ไม่ clean) → Transform: `raw` → `staging` (กำหนด type, dedupe, ล้างข้อความ, กรองแถวเสีย) แล้ว `staging` → `marts` (สร้าง star schema) → Export: เขียนเป็น Parquet แบ่ง partition ตามหมวด → Publish: push dataset #1 ขึ้น Hugging Face
- 2 **Preprocessing (`preprocessing/preprocessing.ipynb`)** — โหลด dataset #1 กลับจาก Hugging Face → ล้าง HTML ในข้อความ → กรองรีวิวสั้นเกินไป → สร้าง label (`sentiment`) และ feature ของข้อความ → รวม feature จาก dimension (ราคา, แบนด์, พฤติกรรมผู้รีวิว, สถิติสินค้า) → แบ่ง train/val/test ตามเวลา → export และ push dataset #2
- 3 **Analytics & Modeling (`analytics/` , `model/`)** — ตอบคำถามกลุ่ม A ด้วย SQL/pandas บน dataset #1 และคำถามกลุ่ม B ด้วยโมเดล scikit-learn (Regression, Classification, Clustering, Topic Modeling, Anomaly Detection) บน dataset #2 พร้อมบันทึกโมเดลที่เทรนแล้วเป็นไฟล์ `.joblib`
- 4 **Dashboard (`dashboard/`)** — Streamlit application ที่ query dataset #1 สดผ่าน DuckDB สำหรับคำถามกลุ่ม A และโหลดผลลัพธ์จากโมเดลที่เทรนไว้แล้ว สำหรับคำถามกลุ่ม B (ไม่ retrain สดในแอป)

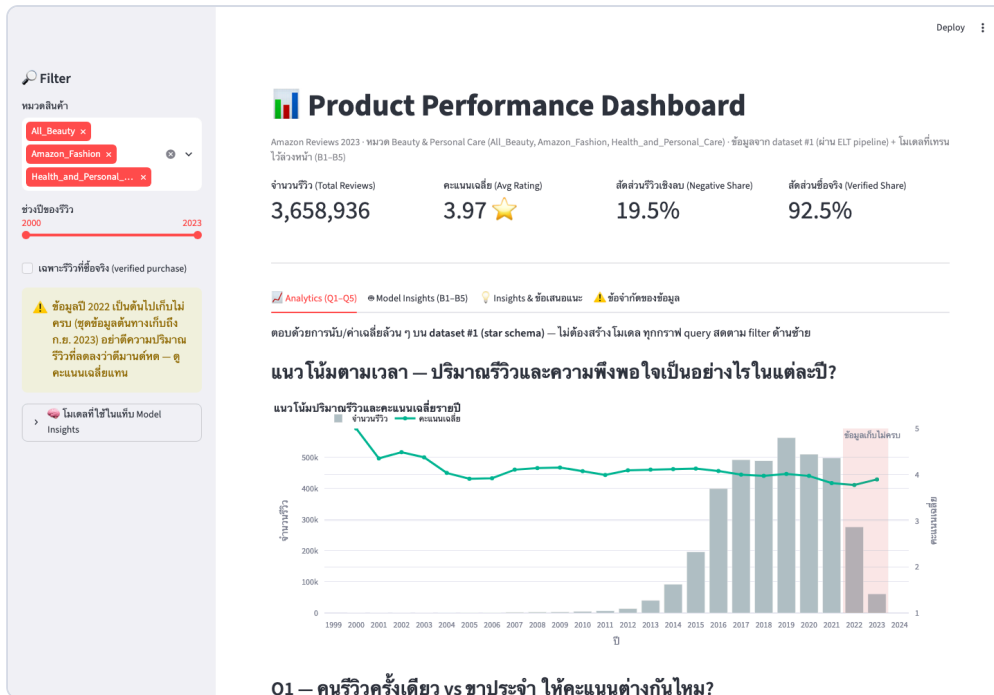
เครื่องมือที่ใช้

DuckDB SQL engine หลัก — query ตรงบนไฟล์ Parquet ได้โดยไม่ต้องโหลดข้อมูลทั้งก้อน เข้า RAM	Python 3.10 ภาษาหลักของทุก stage	Hugging Face Hub เก็บและแจกจ่าย dataset ทั้งดิบและ หลังประมวลผล (public)
pandas / pyarrow อ่าน-เขียนไฟล์ Parquet และจัดการ DataFrame	Jupyter / nbclient รัน notebook แบบ headless (reproducible, ไม่ต้องเปิดหน้าจอ)	scikit-learn โมเดล B1-B5 (Ridge, GradientBoosting, LogisticRegression, KMeans, LDA, IsolationForest)
Streamlit + Plotly สร้าง Interactive Dashboard	joblib บันทึกโมเดลที่เทรนแล้วเป็นไฟล์ ให้ dashboard โหลดใช้ซ้ำโดยไม่ต้อง retrain	Playwright ทดสอบ dashboard อัตโนมัติด้วย headless browser และจับภาพหน้าจอ สำหรับรายงานนี้

หลักการ ELT ไม่ใช่ ETL — ข้อมูลดิบถูก Load เข้า DuckDB ตามสภาพเดิมก่อน (schema `raw`) แล้วค่อย Transform ด้วย SQL ภายในฐานข้อมูล แทนที่จะ transform ก่อนโหลด วิธีนี้ทำให้ตรวจสอบย้อนกลับได้ง่าย (ข้อมูลดิบยังอยู่ครบ) และ DuckDB จัดการข้อมูลระดับหลายล้านแถวได้เร็วกว่าประมวลผลด้วย pandas ล้วน ๆ

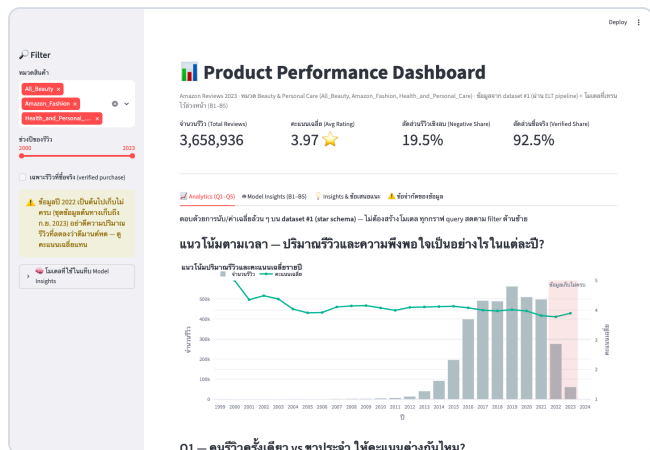
8 Dashboard — ภาพรวมและตัวอย่างการใช้ Filter

สร้างด้วย Streamlit + Plotly มี Filter/Interactive Control 3 รายการ (เลือกหมวดสินค้า, ช่วงปี, เฉพาะรีวิวที่ซื้อจริง) และแบ่งเป็น 4 แท็บ: Analytics (Q1–Q5), Model Insights (B1–B5), Insights & ข้อเสนอแนะ, และข้อจำกัดของข้อมูล

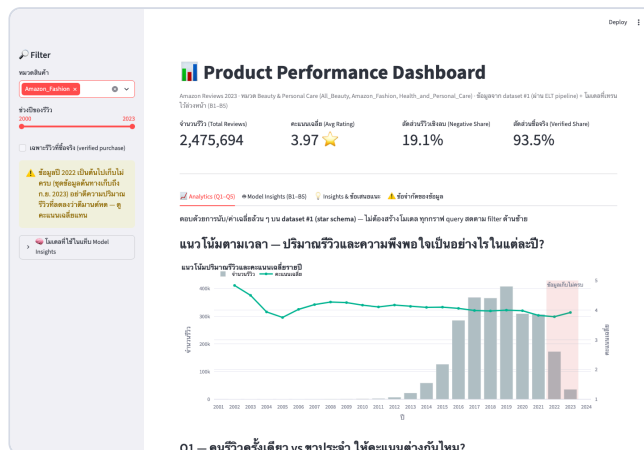


ภาพรวมและ KPI — แถบด้านซ้ายคือ filter ทั้ง 3 รายการ ส่วนบนแสดง Measures หลัก 4 ตัวที่อัปเดตตาม filter แบบเรียลไทม์ ด้านล่างเป็นกราฟแนวโน้มตามเวลา (Time Series) แสงสีแดงช่วงปี 2022–2023 เตือนว่าข้อมูลเก็บไม่ครบ

ตัวอย่างการใช้ Filter — ก่อนและหลังเลือกเฉพาะหมวด Amazon_Fashion

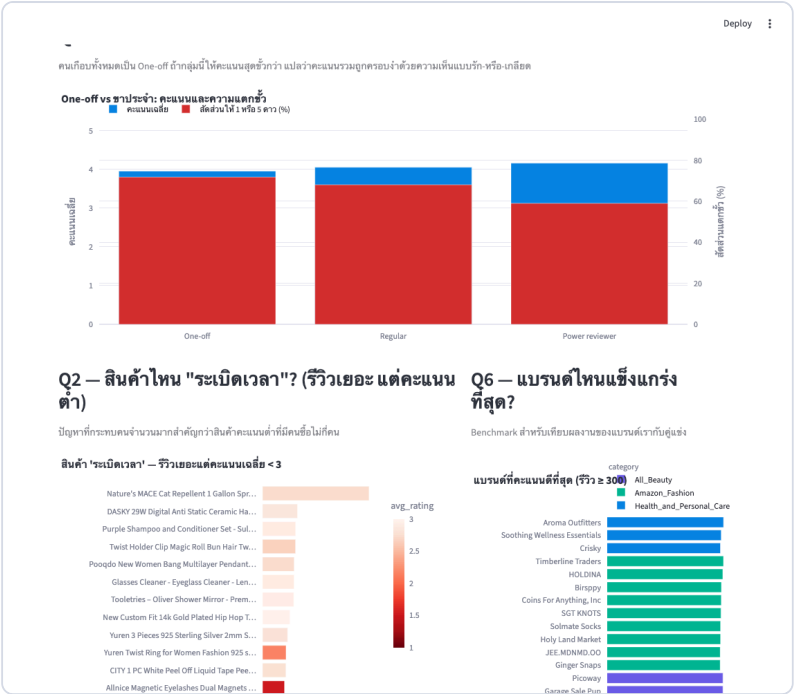


ก่อน filter (ทั้ง 3 หมวด): 3,658,936 รีวิว, คะแนนเฉลี่ย 3.97, verified 92.5%



หลัง filter (เฉพาะ Amazon_Fashion): 2,475,694 รีวิว, คะแนนเฉลี่ย 3.97, verified เปลี่ยนเป็น 93.5% — ทุกกราฟด้านล่างคำนวณใหม่ทันที

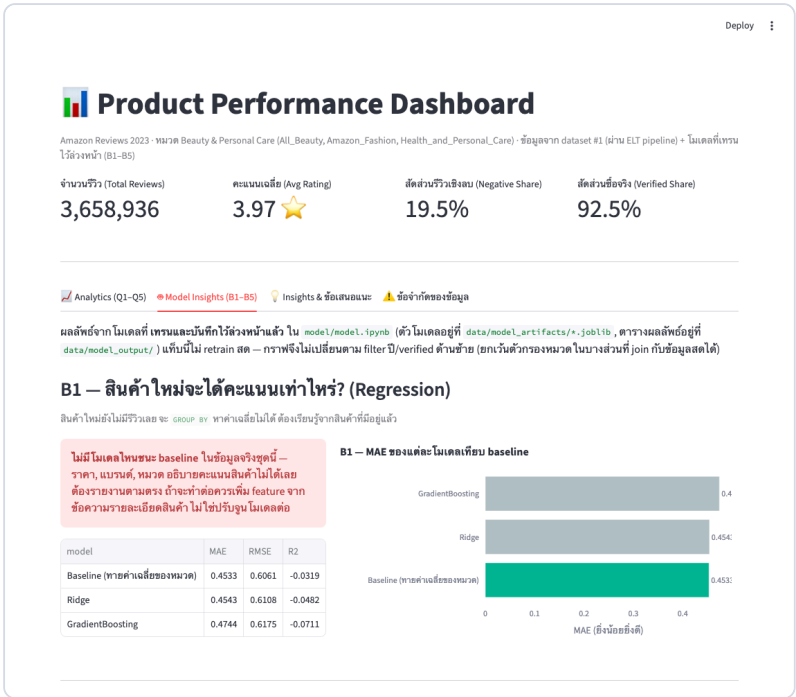
แท็บ Analytics — ตัวอย่างคำถามกลุ่ม A



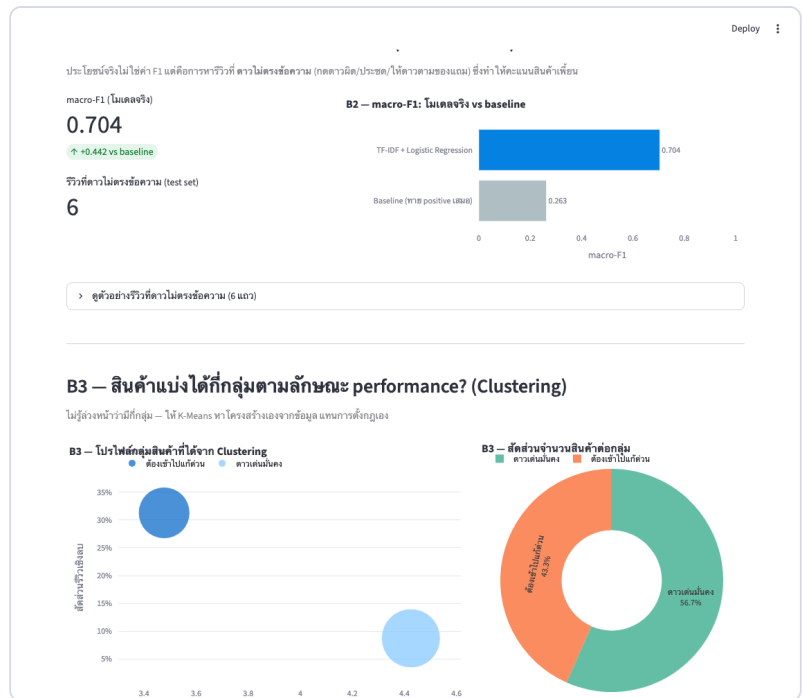


A2 — ราคากับความพึงพอใจ ซ้าย: เปรียบเทียบคะแนนของกลุ่มที่รู้ราคา vs ไม่รู้ราคา (พิสูจน์ MNAR bias) ขวา: คะแนนเฉลี่ยตามช่วงราคา (Bar Chart แบบ ordinal x-axis)

แท็บ Model Insights — ตัวอย่างคำถามกลุ่ม B



B1 — ทำนายคะแนนสินค้าใหม่ (Regression) — ตารางและ Bar Chart เปรียบเทียบ MAE ของ Baseline, Ridge และ GradientBoosting รายงานตามจริงว่า ไม่มีโมเดลไหนชนะ baseline ในข้อมูลชุดนี้



B2 — ข้อความตรงกับดาวไหม (Classification) + B3 — แบ่งกลุ่มสิ้นค้ำ (Clustering) — macro-F1 ของโมเดลจริง (0.704) เทียบ baseline (0.263) และ Bubble/Donut Chart แสดงไปสโกล์กลุ่มสิ้นค้ำที่ K-Means หาได้



B4 — หัวข้อการบ่น (Topic Modeling) + B5 — Anomaly Detection — Stacked Bar สัดส่วนหัวข้อร้องเรียนเทียบข้ามหมวด, แนวโน้มหัวข้อตามปี (Time Series ที่สอง), และ KPI + Horizontal Bar แสดงผลกระทบต่อคะแนนสินค้าหากตัดวีรวิที่น่าสงสัยออก

9 Business Insights และข้อเสนอแนะ

Insight ทั้ง 7 ข้อคำนวณสัดจากข้อมูลภายใต้ filter ที่เลือกไว้บน Dashboard แท็บ "Insights & ข้อเสนอแนะ" (ตัวเลขด้านล่างคือค่าเมื่อเลือกทั้ง 3 หมวดและทุกปี ตัวอย่างหน้าจอก็เก็บนี้อยู่ในหัวข้อ 8)

1 คะแนนรวมถูกครอบงำโดยผู้รีวิวครั้งเดียว

85% ของรีวิวมาจากคนที่รีวิวครั้งเดียวจบ (One-off) และกลุ่มนี้มีสัดส่วนให้คะแนน 1 หรือ 5 ดาวสุดขั้วถึง 72% เทียบกับขาประจำ (Power reviewer) ที่ 59%

ข้อเสนอแนะ: รายงานคะแนนเฉลี่ยควรกำกับฐานผู้รีวิวเสมอ และใช้กลุ่ม One-off + not verified เป็นจุดเริ่มต้นของการตรวจรีวิวปลอม (ดู B5)

2 ข้อมูลราคาขาดหายแบบไม่สุ่ม (MNAR)

ราคาไม่ทราบใน 79% ของรีวิว และสินค้าที่รู้ราคามีคะแนนเฉลี่ยสูงกว่ากลุ่มที่ไม่รู้ราคา อยู่ +0.22 ดาว

ข้อเสนอแนะ: ทุกครั้งที่วิเคราะห์เรื่องราคาต้องรายงานว่ากรองรีวิว ออกไปกี่ % และห้ามเอาข้อสรุปไปอ้างอิงกับสินค้าทั้งหมด

3 รีวิวที่แบบรูปได้รับความเชื่อถือมากกว่าอย่างชัดเจน

สัดส่วนรีวิวที่ได้อย่างน้อย 1 helpful vote สูงกว่าเฉลี่ย +21.2% เมื่อมีรูปแบบ แต่รีวิวที่มีรูปมีอยู่แค่ ~6% ของรีวิวทั้งหมด

ข้อเสนอแนะ: ออกแบบ UX ให้แบบรูปง่ายขึ้น หรือให้ incentive กับรีวิวที่มีรูปแบบ เพื่อเพิ่มสัดส่วนรีวิวคุณภาพสูง

4 ทำนายคะแนนสินค้าใหม่จากราคา/แบรนด์/หมวด เพียงอย่างเดียวไม่ได้ผล

โมเดล Regression (Ridge, GradientBoosting) ไม่มีตัวโน้มนะ baseline (ถายค่าเฉลี่ยของหมวด) เลย

ข้อเสนอแนะ: อย่าใช้ราคา/แบรนด์คัดกรองสินค้าใหม่ก่อนรับเข้าขาย ควรลงทุนเก็บ feature จากคำอธิบายสินค้า หรือรอสะสมรีวิวจริงก่อนประเมิน

5 พบสินค้าที่คะแนนอาจถูก "ดันขึ้น" ด้วยรีวีน่าสงสัย

จาก 100 รีวิวที่โมเดล Anomaly Detection ชี้ว่าน่าสงสัยที่สุด มี 94 สินค้าได้รับ ผลกระทบ และ 48 ชิ้นในนั้นคะแนนจะลดลงถ้าตัดรีวีน่าสงสัยออก (แปลว่าอาจถูกดันคะแนนขึ้น)

ข้อเสนอแนะ: ส่งรายชื่อสินค้ากลุ่มนี้ให้ทีม Trust & Safety ตรวจสอบก่อน — ผลลัพธ์เป็นแค่ "น่าสงสัย" ต้องมีคนตรวจซ้ำเสมอ

6 แต่ละหมวดมีปัญหาเด่นต่างกันชัดเจน

All_Beauty และ Health_and_Personal_Care เเด่นเรื่อง "product, money, waste" (ความคุ้มค่า) ส่วน Amazon_Fashion เเด่นเรื่อง "small, size, fit" (ไซส์/ความพอดี)

ข้อเสนอแนะ: ส่งหัวข้อที่พบบ่อยของแต่ละหมวดให้ทีมพัฒนาสินค้า/ ทีมเขียนรายละเอียดสินค้าแก้ตรงจุด แทนการแก้ปัญหแบบเหมารวมทั้งพอร์ต

7 อย่าตีความปริมาณรีวิวปี 2022-2023 ว่าดีมานด์กำลังหด

จำนวนรีวัวร์่วงจาก ~468K (2021) เหลือ ~58K (2023) ซึ่งเป็นข้อจำกัดของการเก็บ ข้อมูลต้นทาง (เก็บถึงแค่ ก.ย. 2023) ไม่ใช่ความสนใจของลูกค้าที่ลดลงจริง

ข้อเสนอแนะ: รายงานแนวโน้มด้วยคะแนนเฉลี่ยแทนจำนวนรีวิวเมื่อพูดถึง ปี 2022 เป็นต้นไป และตัดปีที่ข้อมูลไม่ครบออกจากรายเปรียบเทียบเชิงปริมาณ

10 การใช้ Generative AI และวิธีตรวจสอบผลลัพธ์

ขอบเขตการใช้งาน

ทีมงานใช้ **Claude (Anthropic) ผ่าน Claude Code** เป็นผู้ช่วยตลอดกระบวนการพัฒนา รวมกลุ่ม:

- ออกแบบและเขียนโค้ด ELT pipeline (Python + SQL บน DuckDB) รวมถึง SQL macro ทำความสะอาดข้อมูลและ view ตรวจสอบคุณภาพข้อมูล
- เขียน preprocessing pipeline, feature engineering และการแบ่ง train/val/test
- เขียนโมเดลตอบคำถามกลุ่ม B ทั้ง 5 ข้อ (scikit-learn) พร้อมออกแบบ baseline สำหรับเปรียบเทียบทุกโมเดล
- เขียน Interactive Dashboard (Streamlit + Plotly)
- จัดทำรายงานฉบับนี้ รวมถึงจับภาพหน้าจอ Dashboard ด้วยการทดสอบอัตโนมัติ

วิธีตรวจสอบผลลัพธ์ (Verification)

เนื่องจากงานส่วนใหญ่สร้างโดย AI ทีมงานจึงยึดหลักตรวจสอบทุกขั้นตอนก่อนใช้งานจริง ดังนี้:

- 1 รับจริงด้วยข้อมูลจริงทั้งหมด** — ทุก notebook/pipeline ถูกรันจนจบบนข้อมูลจริง 3.66 ล้านรีวิว (ไม่ใช่แค่ข้อมูลตัวอย่าง) แล้วตรวจสอบผลลัพธ์ที่ได้ ไม่ใช่แค่ยืนยันว่า "โค้ดรันผ่านไม่ error"
- 2 Automated data-quality checks ฝังในทุก stage** — ELT pipeline มี view ตรวจสอบคุณภาพข้อมูล 10 รายการ (referential integrity ระหว่าง fact กับ dimension, ค่าซ้ำ, ค่านอกช่วงที่เป็นไปได้) ถ้ามีรายการใดไม่ผ่าน ขึ้น export จะหยุดทำงานทันที
- 3 ทดสอบ Dashboard ด้วย Headless Browser จริง** — ใช้ Playwright เปิดแอปและ ตรวจสอบทุกแท็บ ทุก filter จริง แทนการอ่านโค้ดแล้วสันนิษฐานว่าใช้งานได้ ระหว่างการทดสอบ พบและแก้ไขบั๊กจริง 2 จุด: (1) DuckDB connection ที่ใช้ร่วมกันเกิด race condition เมื่อมีการ query ซ้อนกัน ยืนยันด้วยการทดสอบ concurrency 15 เธรดพร้อมกัน — ก่อนแก้พบ error 14/15 ครั้ง หลังแก้ (เปลี่ยนไปใช้ cursor แยกต่อคำสั่ง) เหลือ error 0/15 ครั้ง (2) สคริปต์จับภาพหน้าจอที่ scroll ไปยัง element ผิด แก้ไขจนได้ภาพที่ถูกต้องครบทุกภาพ
- 4 เก็บตัวเลขข้าม 2 แหล่งอิสระ** — ตัวเลขสำคัญที่ปรากฏใน Dashboard (เช่น ส่วนต่างคะแนน +0.22 ดาวจากราคาที่หายไป, ส่วนต่าง helpful-vote +21.2% จากรูปแบบ) ถูกคำนวณซ้ำด้วย SQL แยกต่างหากนอก Dashboard เพื่อยืนยันว่าตรงกัน
- 5 รายงานผลลัพธ์ตามจริงแม้ไม่สวย** — เมื่อโมเดล B1 (Regression) ไม่ชนะ baseline เลยในข้อมูลจริง รายงานฉบับนี้และ Dashboard สะท้อนตรงไปตรงมาแทนที่จะปรับแต่ง หรือเลือกแสดงเฉพาะผลลัพธ์ที่ดี
- 6 มนุษย์เป็นผู้ตัดสินใจในจุดที่มีผลกระทบจริง** — การเปิด dataset/GitHub repo เป็น public, การ push โค้ดขึ้น git, และชื่อ/ขอบเขตของโปรเจกต์ ล้วนผ่านการยืนยันจาก ทีมงานก่อนดำเนินการทุกครั้ง Generative AI ไม่ได้ตัดสินใจเองในจุดที่กระทบความเป็น ส่วนตัวหรือความปลอดภัยของข้อมูล

ข้อจำกัดที่ควรทราบ — แม้จะตรวจสอบหลายชั้น แต่โค้ดและการตีความส่วนใหญ่มาจาก AI ผู้อ่านรายงานควรพิจารณาผลลัพธ์ ประกอบกับ SQL/โค้ดต้นฉบับใน repository เสมอ โดยเฉพาะก่อนนำไปใช้ตัดสินใจทางธุรกิจจริง

11 ลิงก์ไปยัง Source Files และ Dashboard

รายการ	ลิงก์	หมายเหตุ
GitHub Repository (ELT, preprocessing, analytics, model, dashboard ควบ)	github.com/jjentaa/Product-Performance-Analytics-on-Marketplace-Data	Private repository — ต้องได้รับสิทธิ์เข้าถึงจากเจ้าของ repo
Interactive Dashboard (Streamlit Community Cloud)	[รอ URL หลัง deploy — โปรดอัปเดตก่อนส่งรายงานฉบับสมบูรณ์]	รันในเครื่องได้ทันทีด้วย <code>streamlit run dashboard/app.py</code>
Dataset #1 — Star Schema	huggingface.co/datasets/Madnesss/amazon-beauty-star-schema	Public — เข้าถึงได้โดยไม่ต้องใช้ token
Dataset #2 — พร้อมเทรนโมเดล	huggingface.co/datasets/Madnesss/amazon-beauty-analytics-ready	Public — เข้าถึงได้โดยไม่ต้องใช้ token
เอกสารคำถามธุรกิจฉบับเต็ม (พร้อม SQL ทุกข้อ)	<code>docs/business_questions.md</code> (ใน repo)	รวมทั้ง 10 คำถาม A1–A5, B1–B5 พร้อม SQL ที่รันได้จริง

โครงสร้างโฟลเดอร์หลักใน Repository

elt/ ELT pipeline (extract, load, transform, export, publish)	preprocessing/ Preprocessing + EDA notebook	analytics/ Notebook ตอบคำถามกลุ่ม A
model/ Notebook โมเดล B1–B5	dashboard/ Streamlit application	docs/ เอกสารคำถามธุรกิจ + ER diagram

— จบรายงาน —

รายงานฉบับนี้จัดทำร่วมกับ Generative AI (Claude, Anthropic) รายละเอียดวิธีตรวจสอบผลลัพธ์อยู่ในหัวข้อ 10